

Characterizing Speculative Decoding Dynamics for Large Language Models on Consumer-Class GPUs

Samsu Sempena Zhuohan Cui

ICECCME 2026

15–17 October 2026, Bali, Indonesia

The University of Sydney

Motivation

Decoding is sequential, and that is the bottleneck

- Autoregressive decoding needs **one full forward pass of the target model per token**.
- Single-stream, interactive latency is therefore bound by *memory traffic*, not by arithmetic.
- Speculative decoding is the standard fix — and it is reported to be a large, free win.

The practitioner's problem

Almost all published gains are measured on **data-centre** hardware. On a single RTX-class GPU, what actually happens?

Open deployment questions

Which draft size?	0.5B? 1.5B?
Which proposal length?	$k = 4?$ 8? 16?
Greedy or sampling?	speed vs. quality
Does it transfer?	desktop → laptop

Speculative decoding in one slide

Draft $\rightarrow$ verify $\rightarrow$ commit

1. A small **draft** model proposes k tokens autoregressively.
2. The large **target** verifies all k in **one** parallel forward pass.
3. Commit the accepted prefix; add one correction token on the first mismatch.

Modified rejection sampling keeps the output distribution *identical* to the target's.

Leviathan et al. 2023; Chen et al. 2023

Per-cycle latency

$$L = \frac{T_{\text{draft}} + T_{\text{verify}}}{\tau}, \quad T_{\text{draft}} = k \cdot t_{\text{draft step}}$$

$\tau \in [1, k+1]$ accepted tokens per cycle.

- Numerator grows **linearly** in k .
 - Denominator τ saturates as acceptance α falls.
- $\Rightarrow$ There must be a k beyond which it *loses*.

What is missing, and what we ask

Three gaps in the literature

- No cross-model *and* cross-hardware validation at the consumer tier.
- Speedups reported as bare means — no sample- or seed-level dispersion.
- Rarely any statistical test against a matched autoregressive baseline.

This is a *characterization* paper, not a new decoding algorithm.

Research questions

- RQ1** How much end-to-end speedup on consumer RTX hardware?
- RQ2** How do k and acceptance *jointly* set net latency?
- RQ3** How sensitive is the trade-off to deterministic vs. stochastic decoding?
- RQ4** Do rankings and gains transfer RTX4090 → RTX5090-laptop?

Protocol

Experimental protocol

Models (same-family pairings)

- Qwen2.5-3B-Instruct $\leftarrow$ 0.5B draft
- Qwen3-4B-Instruct $\leftarrow$ 0.6B draft
- FP16, matched tokenizers, KV cache

Hardware

- RTX4090 desktop, 1008 GB/s, 450 W
- RTX5090 laptop, 896 GB/s, ≤ 150 W

Data — 1000 samples, frozen manifests:
GSM8K 300 · MMLU 500 · CNN/DM 200

Fixed-policy grid

$k \in \{4, 8, 16\} \times \{\text{deterministic}, \text{stochastic}\}$
 $\Rightarrow$ 6 configurations **per family per device**,
each against a **regime-matched** AR baseline.

Deterministic: $T=0$, $\text{top-}p=1$. Stochastic:
 $T=0.7$, $\text{top-}p=0.9$.

Controls

Seed 42 primary; repeat seeds 123/999 at
 $k=4$. Frozen sample-ID manifests, identical
prompts and length limits, warm-up before
measurement, serial execution.

Results

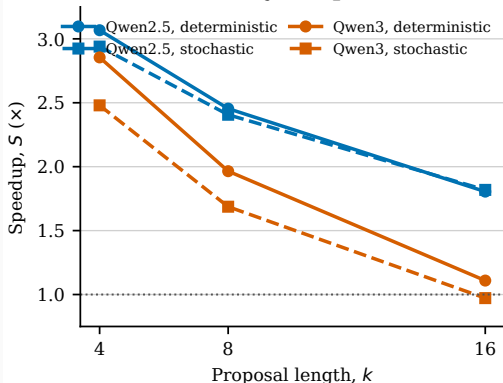
RQ1 — On the RTX4090, it works

Speedup S	$k = 4$		$k = 8$		$k = 16$	
	det	stoch	det	stoch	det	stoch
Qwen2.5 (3B $\leftarrow$ 0.5B)	3.0674	2.9394	2.4542	2.4063	1.8050	1.8185
Qwen3 (4B $\leftarrow$ 0.6B)	2.8548	2.4798	1.9648	1.6869	1.1091	0.9719
Qwen3 acceptance α	0.336	0.267	0.238	0.192	0.129	0.108
Qwen3 block B_{eff}	1.33	1.06	1.86	1.50	1.96	1.64

- 11 of 12 configurations beat baseline.
- Both families peak at **deterministic $k=4$** .
- Qwen3 stochastic $k=16$ is the one loss (**0.97 $\times$**).
- Long proposals can erase the gain entirely.

RQ2 — Bigger blocks, slower generation

(a) Model-family comparison on RTX4090



Mean speedup, $k=4 \rightarrow k=16$

- Qwen2.5: 2.9164 $\rightarrow$ 1.8118
- Qwen3: 2.6673 $\rightarrow$ 1.0405

Token timing degrades too

(Qwen3, RTX4090, deterministic)

- TTFT: 132.6 $\rightarrow$ 349.6 ms
- TPOT: 41.2 $\rightarrow$ 94.2 ms/token

B_{eff} rises while latency worsens: the overhead inflection arrives before $k=8$.

RQ3 — Deterministic decoding is the safer default

Qwen3, RTX4090	$k=4$	$k=8$	$k=16$
Deterministic S	2.8548	1.9648	1.1091
Stochastic S	2.4798	1.6869	0.9719
Advantage	+0.3750	+0.2779	+0.1372

- Greedy drafting agrees with greedy verification more often $\Rightarrow$ higher α (0.336 vs. 0.267 at $k=4$).
- The advantage *shrinks* with k — once acceptance is low, regime hardly matters.

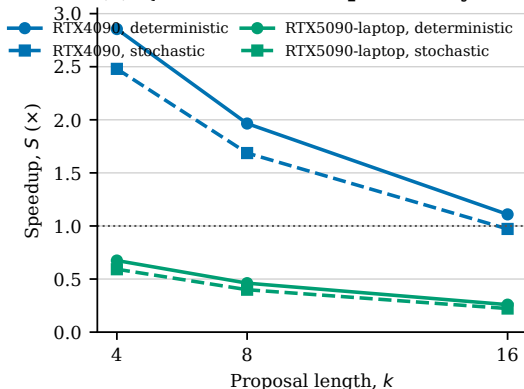
Not universal

Qwen2.5 follows the same ordering at $k=4$ and $k=8$, but reverses slightly at $k=16$ (1.8050 det vs. 1.8185 stoch).

Takeaway. If you need sampling, expect to pay $\sim 13\%$ of your speedup for it at low k .

RQ4 — The same policy, a different GPU, and it all inverts

(b) Qwen3 hardware portability



Qwen3 on RTX5090-laptop

Config	S	α
$k=4$, det	0.6743	0.335
$k=4$, stoch	0.5924	0.285
$k=8$, det	0.4613	0.254
$k=8$, stoch	0.3995	0.227
$k=16$, det	0.2588	0.180
$k=16$, stoch	0.2222	0.154

- All six are slower than baseline.
- Acceptance *equals or beats* RTX4090 at every k — the loss is in execution cost, not the algorithm.
- Ordering survives: low k , deterministic.

Is the effect real? Statistics and stability

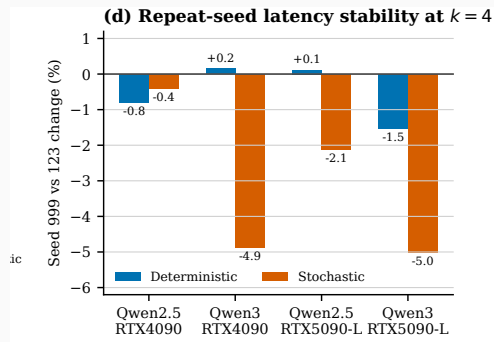
Paired one-sided t -tests, $H_0: S \leq 1$,
paired by sample, Holm–Bonferroni across 6
configs (Qwen3, RTX4090, $n=1000$):

Config	d_S	p_S
det $k=4, 8, 16$	1.18 / 1.10 / 0.29	all $< 10^{-18}$
stoch $k=4, 8$	1.24 / 1.13	both $< 10^{-179}$
stoch $k=16$	-0.08	0.996

Dispersion is large. Per-sample S (det):

2.7561 ± 0.5950 at $k=4 \rightarrow 1.3549 \pm 0.6533$ at
 $k=16$.

At $k=16$ the standard deviation is half the mean —
prompt-to-prompt variation dominates.



Seed 999 vs. 123 at $k=4$: deterministic within
1.5%; stochastic 0.4%–5.0%.

Speed is not free: quality on CNN/DailyMail

Deterministic, $n=200$	ROUGE-L	BLEU	BERTScore F1
Autoregressive baseline	19.58	3.20	0.793
Speculative, $k=4$	11.50	1.39	0.710
Speculative, $k=16$	14.92	2.04	0.741

- All three metrics agree: a real quality cost on the summarization task.
- It **partially recovers** as k grows — moving *against* the speed trend.

The fastest configuration is also the weakest on summarization. There is a genuine speed–quality frontier here, not free acceleration.

Wrap-up

What we can and cannot claim

Hypotheses

- H1 $S > 1$ — **supported** on RTX4090 (11/12), **rejected** on RTX5090-laptop.
- H2 $k=8 > k=4$ — **rejected**; speedup falls monotonically.
- H3 B_{eff} up, latency worse — **supported**.
- H4 Deterministic faster — **mostly supported** (one reversal).

Threats to validity

- One PyTorch/Transformers stack; results do not extrapolate to other runtimes or GPUs.
- Same-family Qwen pairings only.
- No long-context, safety, or factual-consistency evaluation.
- Repeat seeds cover $k=4$ only.
- CUDA-graph capture differs across run families — reported as context, *not* as a measured cause.

Policy ranking is portable.

Absolute speedup is not.

Takeaways

1. **Use a short proposal.** On both consumer GPUs and both model families, $k=4$ dominates $k=8$ and $k=16$ — the overhead inflection arrives earlier than expected.
2. **Prefer deterministic decoding** at matched k ; sampling costs roughly 13% of the speedup at $k=4$.
3. **Benchmark on your target stack.** An RTX4090 gave up to $3.07\times$; the same code on an RTX5090 laptop was *sub-baseline everywhere*, with unchanged acceptance.
4. **Report dispersion and quality, not just a mean speedup.** Per-sample spread is large, and summarization quality moves against the speed trend.

Frozen manifests, matched baselines, and raw per-sample artifacts support reproduction.

Backup slides

Backup — full Qwen3 RTX4090 grid

Config	S	α	B_{eff}	T_{mean} (s)	TTFT (ms)	TPOT (ms)
$k=4$, det	2.8548	0.3362	1.33	4.8881	132.57	41.23
$k=4$, stoch	2.4798	0.2672	1.06	5.6196	134.54	45.51
$k=8$, det	1.9648	0.2378	1.86	7.1024	204.99	58.76
$k=8$, stoch	1.6869	0.1916	1.50	8.2612	207.82	64.67
$k=16$, det	1.1091	0.1292	1.96	12.5817	349.58	94.24
$k=16$, stoch	0.9719	0.1082	1.64	14.3390	354.82	106.03

Target: Qwen3-4B-Instruct; draft: Qwen3-0.6B-Instruct; $n=1000$.

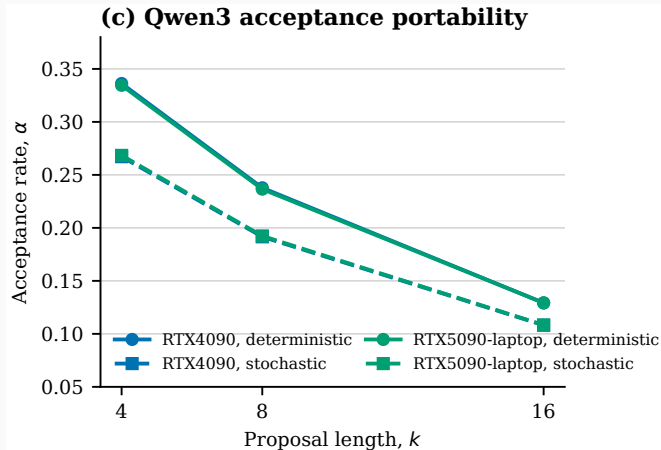
Backup — full Qwen2.5 RTX4090 grid

Config	S	α	B_{eff}	T_{mean} (s)	TTFT (ms)	TPOT (ms)
$k=4$, det	3.0674	0.4212	1.66	3.7210	102.35	29.06
$k=4$, stoch	2.9394	0.4080	1.61	3.9631	105.26	30.89
$k=8$, det	2.4542	0.3515	2.72	4.6508	162.76	36.73
$k=8$, stoch	2.4063	0.3444	2.67	4.8411	165.95	38.04
$k=16$, det	1.8050	0.2539	3.76	6.3234	277.43	49.34
$k=16$, stoch	1.8185	0.2516	3.73	6.4059	278.85	49.81

Target: Qwen2.5-3B-Instruct; draft: Qwen2.5-0.5B-Instruct; $n=1000$.

Higher α and B_{eff} than Qwen3 at every k .

Backup — acceptance portability



Qwen3 acceptance on the laptop is identical at $k=4$ (0.335 vs. 0.336) and *higher* at $k=8$ and $k=16$ (0.254 vs. 0.238; 0.180 vs. 0.129). The algorithm behaves at least as well; the **runtime economics** differ.

Backup — success criteria and metrics

Stated success criteria

- Primary: mean $S \geq 1.3\times$ with Holm–Bonferroni-corrected $p_S < 0.05$.
- Primary: absolute quality drop ≤ 1.0 point on GSM8K and MMLU, deterministic regime.
- Secondary: best (k , regime) under quality constraints; dispersion; seed stability; cross-hardware ranking transfer.

Key metric definitions

- S = baseline latency / speculative latency
- α = accepted / proposed draft tokens
- B_{eff} = accepted draft tokens / verify steps
- TTFT = first token time – request start
- TPOT = (completion – first token) / output tokens
- d_S = Cohen's d , paired baseline vs. speculative latency

Backup — hardware profiles

Component	RTX4090 desktop	RTX5090 laptop
GPU memory	24 GB GDDR6X	24 GB GDDR7
Memory bandwidth	1008 GB/s	896 GB/s
CUDA cores	16384	10496
Architecture	Ada Lovelace	Blackwell
Power profile	450 W TGP class	up to 150 W TGP (OEM-dependent)
Software stack	PyTorch + Transformers, pinned per run family	

The laptop part is the newer architecture but has lower bandwidth and roughly **one third of the power budget**.